

Pynapple, a toolbox for data analysis in neuroscience

Guillaume Viejo, Daniel Levenstein, Sofia Skromne Carrasco, Dhruv Mehrotra, Sara Mahallati, Gilberto R Vite, Henry Denny, Lucas Sjulson, Francesco P Battaglia, Adrien Peyrache 

Montreal Neurological Institute and Hospital, McGill University, Montreal, QC, Canada • MILA – Quebec IA Institute
• Departments of Psychiatry and Neuroscience, Albert Einstein College of Medicine, Bronx, NY • Donders Institute for Brain, Cognition and Behaviour, Radboud University, 6525AJ Nijmegen, The Netherlands

Reviewed Preprint

Revised by authors after peer review.

About eLife's process

Reviewed preprint version 2

August 25, 2023 (this version)

Reviewed preprint version 1

May 17, 2023

Sent for peer review

January 24, 2023

Posted to preprint server

December 7, 2022

 https://en.wikipedia.org/wiki/Open_access

 Copyright information

Abstract

Datasets collected in neuroscientific studies are of ever-growing complexity, often combining high dimensional time series data from multiple data acquisition modalities. Handling and manipulating these various data streams in an adequate programming environment is crucial to ensure reliable analysis, and to facilitate sharing of reproducible analysis pipelines. Here, we present Pynapple, the PYthon Neural Analysis Package, a lightweight python package designed to process a broad range of time-resolved data in systems neuroscience. The core feature of this package is a small number of versatile objects that support the manipulation of any data streams and task parameters. The package includes a set of methods to read common data formats and allows users to easily write their own. The resulting code is easy to read and write, avoids low-level data processing and other error-prone steps, and is open source. Libraries for higher-level analyses are developed within the Pynapple framework but are contained within in a collaborative repository of specialized and continuously updated analysis routines. This provides flexibility while ensuring long-term stability of the core package. In conclusion, Pynapple provides a common framework for data analysis in neuroscience.

Highlights

- An open-source framework for data analysis in systems neuroscience.
- Easy-to-use object-oriented programming for data manipulation.
- A lightweight and standalone package ensuring long-term backward compatibility.

eLife assessment

This paper introduces the python software package Pynapple and a separate package of more advanced routines (Pynacollada) to the Neuroscience/Neural Engineering community. Pynapple provides a set of data objects and methods that have the potential to simplify data analysis for neural and behavioral data types. This represents a **valuable** contribution to the field. With more examples and as a live coding notebook, the evidence was judged to be **compelling**.

Introduction

The increasing size of datasets across scientific disciplines has led to the development of specific tools to store ¹, ², analyze ³, and visualize ⁴ them. While various programming environments such as Matlab and R have long been commonly used in data science, Python has progressively become one of the most popular programming languages ⁵. This is due to its open nature, large community-driven development, and versatility of usage. As with virtually all other scientific fields, neuroscience faced the challenges of handling and analyzing large datasets by rapidly developing a wide range of specialized tools to deal with each of these types of data ⁶–¹¹ and corresponding analyses.

In systems neuroscience, calcium imaging and high-density electrophysiology make it possible to simultaneously monitor the activity of an increasingly large number of neurons ¹², ¹³. Often, this is combined with simultaneous behavioral recordings. As in all other fields, this has required the development of specific pipelines to process ¹⁴–²⁵ and store ²⁶, ²⁷ the data. Despite this rapid progress, data analysis often relies on custom made, lab-specific code, which is susceptible to error and can be difficult to compare across research groups. While several toolboxes are available to perform neuronal data analysis ⁸–¹¹, ²⁸–³⁰ (see ref. ³¹ for review), most of these programs focus on producing high-level analysis from specified types of data, and do not offer the versatility required for rapidly-changing analytical methods and experimental methods. Users can decide to use low-level data manipulation packages such as Pandas, but in that case, the learning curve can be steep for users with low, if any, computational background.

The key challenge for scientific code is balancing the need for flexibility and stability. This is especially true of science because results should be reproducible (between labs, between the past and the future, and between different experimental setups) while keeping up with rapidly changing requirements (e.g., due to new kinds of data, theories, and analysis methods). To meet these needs, we designed Pynapple, a general toolbox for data analysis in systems Neuroscience with a few principles in mind.

The first property of such a toolbox is that it should be object-oriented, organizing software around data. This makes the programming environment very efficient for data analysis, particularly in systems neuroscience where data streams can be of very different types. For example, to compute the rate of an event, one can write a function that takes an array of event times and divides the number of elements by the time between the first to the last event. However, this approach neglects to consider that the appropriate epoch in which to calculate the rate could start earlier, or end later, than the first or last event. Addressing these concerns requires another argument, which defines the boundaries of the epoch on which the rate should be computed. Overall, this approach is error prone. The epoch boundaries and event times must be stored in the same time unit and with the same reference (i.e., simultaneous time 0) and the rate function itself can be erroneously called with arrays storing another type of data. In contrast, an object which is

specifically designed to represent a series of event times can ameliorate these concerns. For example, it can be created from a specific data loader that ensures proper definition of time units and support epochs (i.e., true beginning and end of the observation time). It will then be immune to the arithmetic operations that can change the values of a generic array (for example an addition that is misplaced in the code). Further, the object can be endowed with a rate property that is specifically written for this object, reducing the odds of a coding error. While this approach may discourage users who are not familiar with this type of coding, the benefit far exceeds the effort of learning object-oriented programming, especially if the naming of the methods and properties is explicit.

Another property of an efficient toolbox is that a small number of objects could virtually represents all possible data streams in neuroscience, instead of objects made for specific physiological processes (e.g. spike trains). This ensures that the same code can be used for various datasets and eliminates the need of adapting the structure of the package to handle rare or yet-to-be-developed data types. Then, these objects should then be able to interact via a small number of basic and foundational operations, which are sufficient for most analyses. This allows users to quickly write new code for new use-cases, and easily understand and adapt code written by others, as the same methods can be used for any kind of data.

The toolbox should be able to load common data storage types, and the flexibility to create loaders for future and custom/lab-specific data. It should also support the development of yet-unknown, lab-specific, and specialized analysis methods. In other words, the customization of the package to adapt to any dataset should happen at the input stage and the development of high-level analytical methods should take place outside the core package. The properties listed above ensure the long-term stability of a toolbox, a crucial aspect for maintaining the code repository. Toolboxes built around these principles will be maximally flexible and will have the most general application.

In this paper we introduce the Python Neural Analysis Package (Pynapple), designed with these axioms in mind. The core of Pynapple is five versatile timeseries objects, whose methods make it possible to intuitively manipulate and analyze the data. We show how Pynapple can be used with most raw neuroscience data types to produce the most common analyses used in contemporary neuroscience. Additionally, we introduce Pynacollada, a collaborative repository for higher level analyses built from the basic functionality provided by Pynapple. A complete neuroscience data analysis pipeline using a common language supports open and reproducible code. As all users are also invited to contribute to the Pynapple ecosystem, this framework also provides a foundation upon which novel analyses can be shared and collectively built by the neuroscience community.

Core features of Pynapple

At its core, Pynapple is object-oriented. Because objects are designed to be self-contained and interact with each other through well-defined methods, users are less likely to make errors when using them. This is because objects can enforce their own internal consistency, reducing the chances of data inconsistencies or unexpected behavior. Overall, object-oriented programming is a powerful tool for managing complexity and reducing errors in scientific programming. Pynapple is built around only five objects that are divided into three categories: two objects represent event timestamps (one or several), two represent time-varying data (one or several time series at the same sampling times), and one represents time epochs. Raw or pre-processed data are loaded into these objects in the coding environment (**Fig. 1** [↗](#)). The data loaders ensure that all loaded objects have the same time base. Hence, once objects are constructed, the user does not have to remember properties of the data such as the sampling frequency or alignment of data indices to clock time. Then, these objects can be manipulated with their own methods (i.e., object-specific functions). A large majority of data manipulations needed for most users can be achieved with a small number of methods. From there, Pynapple offer some foundational analyses, such as cross-correlation of event times. On top of this, the user may write analytical code that is project specific.

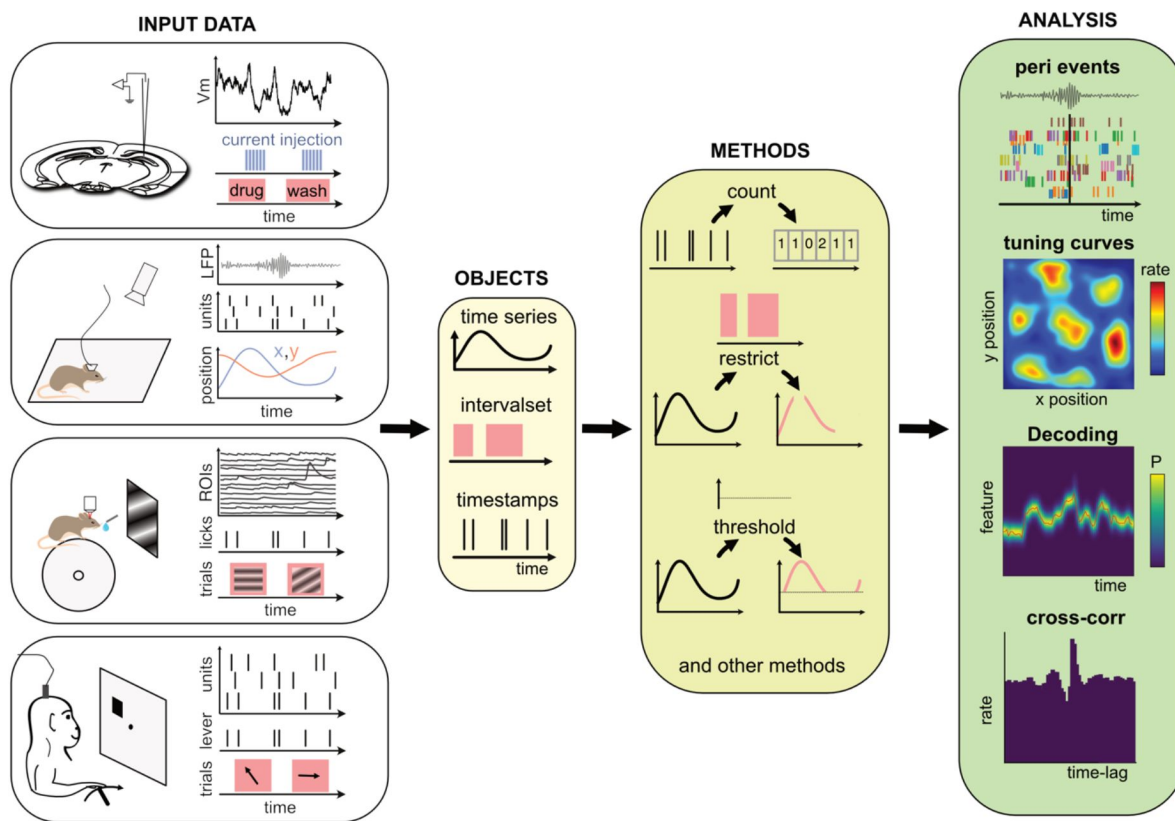


Fig. 1

Data analysis with the Pynapple package.

Left, any type of input data can be loaded in a small number of core objects. For example (from top to bottom): intracellular recordings in slice during which current is injected and drug is applied to the bath solution; extracellular recordings in freely moving mice whose position is video-tracked; calcium imaging in head-fixed mice during presentation of different visual stimuli and delivery of precisely timed rewards; extracellular recordings in non-human primates during the execution of cognitive tasks. *Middle*, object-specific methods allow the user to perform a wide variety of basic operations and to manipulate the data manipulations. *Right*, at a higher level, the package contains a set of foundational analysis methods such as (from top to bottom) peri-event alignment of the data (top), 1- and 2-D tuning curves, 1- and 2-D decoding; auto- and cross-correlation of event times (e.g., action potentials). These methods depend only on a few, commonly used, external packages.

The most basic objects are timestamps (*Ts*), which are typically used for any discrete events, for example spike or lick times. The timestamped data (*Tsd*) object holds timestamps and associated data associated with each timestamp. For example, this object is used to represent an animal's position in its environment, electroencephalogram data, or average calcium fluorescence as a function of time. Two objects were designed to represent groups of *Ts* and *Tsd*, namely *TsGroup* and *TsdFrame*. The main difference between the two objects is that *TsdFrame* has common timestamps for all the data (and therefore, all data have the same number of samples). *TsGroup* is more generic as each element has its own timestamps. These objects are typically used for ensembles of simultaneously recorded spike trains (*TsGroup*) or simultaneously acquired calcium fluorescence (*TsdFrame*). They are useful when operations need to be performed on a common time basis, for example binning multiple spike trains. Note however that they can be used for many other data types, for example the position of the animal (*TsdFrame*). Last, *IntervalSet* objects represent time epochs, for example the start and end times of intervals in which the animal is running.

Pynapple is built with objects from the Pandas library [5](#). As such, Pynapple objects inherit the long-term consistency of the code and the computational flexibility from this widely used package. Specifically, a *Tsd* object is an extension of (or “inherits” in object-orienting programming) Pandas *Series* object and *TsdFrame* of Pandas *DataFrame* object. A *TsGroup* is a child of *UserDict*, a built-in python object for inheriting dictionaries. Finally, *IntervalSet* inherits Pandas *DataFrame*. Timestamps are by default in units of seconds but can be readily converted to other time units using the `as_units` method in any object.

Pynapple objects have a limited number of core methods (**Fig. 2A**), which form the foundation of further operations. These operations provide a general framework by which users can manipulate the timestamps and their corresponding values as needed for analysis. For example, the time series objects have built-in methods: `value_from`, which gets the value from one time series object at the (closest) timestamps from another; `restrict`, which “restricts” a time series object; extracting only the data contained within a set of time intervals defined by an *IntervalSet* object; `count`, which counts the number of timestamps from a time series object in windows of a given bin size; `threshold`, which applies a threshold to the data within a *Ts* or *Tsd* object and returns a *Tsd* containing the data above or below the threshold. All operations can be restricted to a given epoch, specified by an *IntervalSet*.

Furthermore, all objects have a `time_support` property, which keeps track of the time interval over which the data is valid. The time support is an *IntervalSet* object that is attached by default to *Ts*, *Tsd*, *TsdFrame* and *TsGroup* objects. This is a crucial property as, otherwise, it is impossible to know whether periods without data correspond to an epoch during which the underlying event was not observed or because this period has previously been excluded by a `restrict` method.

In addition to the ability to restrict methods of time series objects, the *IntervalSet* object has methods for logical operations on combinations of *IntervalSets*, all returning other *IntervalSets* (**Fig. 2B**): `intersect`, which returns the set intersection of two *IntervalSet* objects; `union`, which returns the set union of two *IntervalSets*; `set_diff`, which returns the set difference of two *IntervalSet*; `drop_short_intervals`, `drop_long_intervals`, which eliminate interval subsets that are shorter or longer than a desired duration; and `merge_close_intervals`, which merge intervals that are closer in time than a given duration.

Many experiments in neuroscience are based on trials, each associated with different conditions. *IntervalSets* are perfectly suited for this, as one *IntervalSet* can represent all start and end times of trials. The nature of each trial (e.g., left/right, correct/error) can be stored as a third column within the *IntervalSet* dataframe object. Thus, subsets of trials can be easily selected to restrict data of interest on the corresponding epochs. An alternative approach is to store different *IntervalSets* for different types of trials.

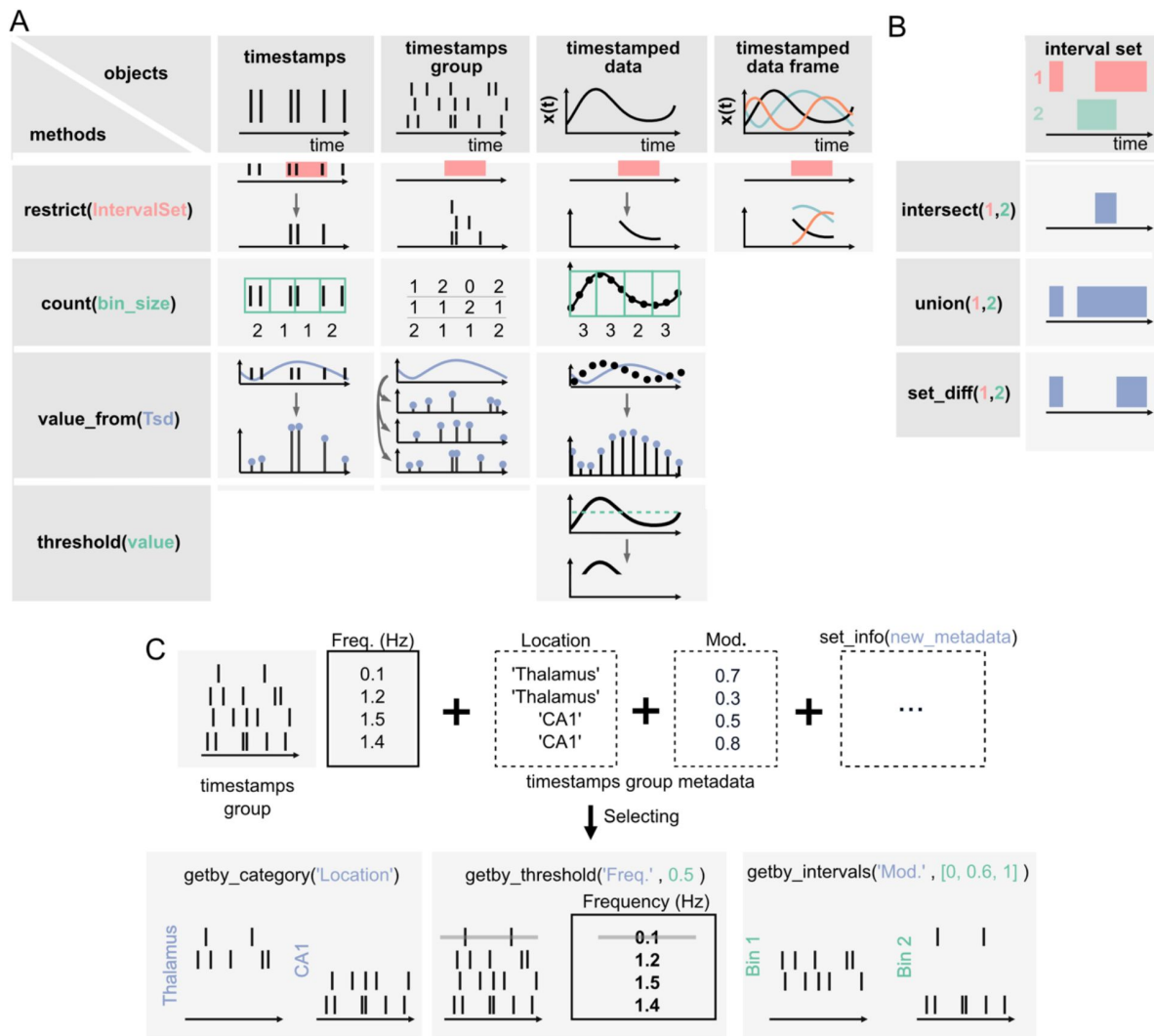


Fig. 2

Core methods of the Pynapple objects.

A) Methods of Timestamps (*Ts*) and Timestamped Data (*Tsd*) objects. The same methods can be called for different objects, leading to qualitatively similar results. For example, `object.restrict(intervalSet)` returns an object now defined on the intersection of its original time support and the input *IntervalSet*. Objects can be any of the timestamps and timestamped data objects. These methods can be called with only one argument, as shown here, since the default parameters are typically the same for most analyses. Yet the methods include additional arguments for more specific operations. B) Logical operations on pairs of *IntervalSet* objects to compute (from top to bottom) the intersection, union, and difference between epochs. These operations are commonly used to analyze data during specific epochs in a combinatorial manner, such as “exploration period AND running speed is above 5cm/s NOT left arm”. C) Methods of *TsGroup* objects. Each timestamp is associated by default with its occurrence rate. Additional custom metadata such as recording location can be added. These metadata can then be used to select and filter timestamps using `getby_category` for discrete labels, `getby_threshold` or `getby_intervals` for numerical values.

In addition to the ability to apply any methods of the *Ts* object to its members, *TsGroup* has a set of methods to calculate and store metadata about the elements of the group (**Fig. 2c**). For example, one can store and retrieve the anatomical structure from which a neuron was recorded, or the result from downstream analysis, perform operations on each element, and filter by various properties. These methods allow the user to, for example, calculate, store, and compare the properties of multiple neurons in a population. Additional methods for all objects are extensively documented in the documentation, and examples for usage are given in the tutorials.

While there are relatively few objects, and while each object has relatively few methods, they are the foundation of almost any analysis in systems neuroscience. However, if not implemented efficiently, they can be computationally intensive and if not implemented accurately, they are highly susceptible to user error. The implementation of core features in Pynapple addresses the concerns of efficiency and accuracy. Crucially, all units are indexed by seconds across the entire package, which limits the need for users to account for indexing and alignment between different streams of data at different sampling rates. For example, a user can simply use `spikes.value_from(position)` to get the animal's position at each spike time, rather than costly and error-prone routines in which a user needs to identify matching indices for the corresponding timestamps across arrays containing spikes and behavioral information. Another common issue in data analysis is to analyze two timeseries that are not recorded at the same sampling rate. Once data are loaded in the same time base (i.e., the same time 0), they can keep their original sampling times. Using the function `value_from` from one object with the other object as argument will provide two time series with the same number of samples and the same sampling times, which will simplify further analyses. However, this means it is essential that all objects are loaded in the same time base for these methods to function correctly. Pynapple anticipates this by providing a customizable data loader, ensuring time bases are always loaded correctly.

Importing data from common and custom pipelines

The proliferation of experimental methods has come with a proliferation of data formats, as well as the need to rapidly develop new formats that meet new experimental needs. Usually, these data formats are dependent on the software that was used to preprocess the raw data, making them difficult to load for further analysis. Additionally, an experimental setup can generate multiple streams of data that are saved within multiple files of various types. Thus, a universal toolbox should be able to load popular data formats into a common framework and offer the user the ability to write functions to load their own data types.

To ease the process of loading and synchronizing data from various streams, Pynapple includes an I/O layer that allows the user to load multiple types of datasets and write them to a common format for further analysis and sharing. The primary way by which a user interacts with the I/O layer is an object that represents an experimental session, with the properties of the object being the various time series. This I/O object is created by calling the function `load_session`, which will load all data associated with that session (**Fig. 3A**). For example, calling `load_session` for an *in vivo* electrophysiology recording would return an object called `data`, which will have properties `data.spikes`, `data.position`, and `data.epochs` which respectively store a *TsGroup*, containing the spike times, a *TsdFrame* containing the position of the animal, and an *IntervalSet* containing the times when the animal is on the track. With this object-oriented I/O method, the user can interact with the various data streams associated with a given experimental session and load multiple sessions at once without the risk of mixing data as each time series is attached to only one I/O object.

Data synchronization is the crux of any analysis pipeline. The `load_session` function is thus a crucial step in using the package. For unsupported data types, it is the responsibility of the users to design the preprocessing scripts that align the data streams in the same absolute time base. The

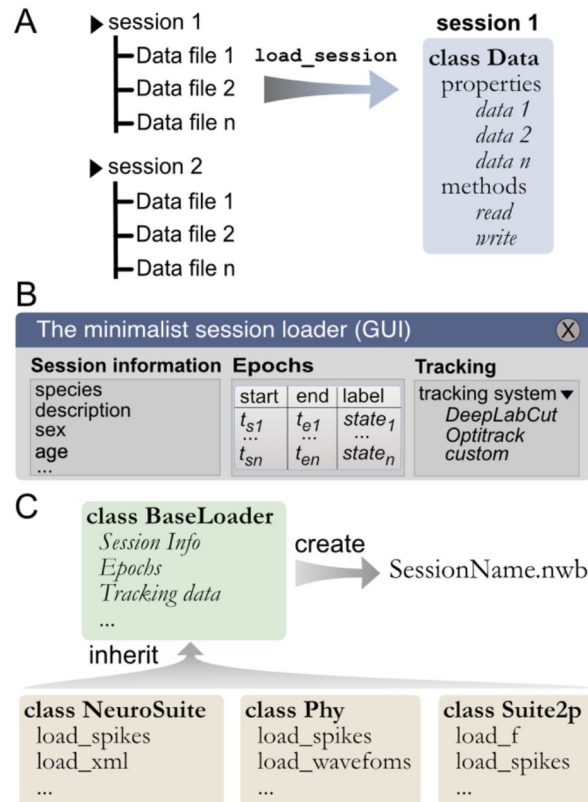


Fig. 3

Built-in and customizable loading function for Pynapple.

A) Data is originally organized as separate files in a folder. A built-in or custom-made `load_session` function is called to load the data into a `Data` class. B) Data can be loaded through a customizable GUI to enter all relevant information regarding the experiment, for example animal strain, among others. The main epochs of the recording (e.g., behavioral states, stimuli category, etc.) can be loaded from standard tabular data files (such as CSV). Behavioral tracking data extracted from various common systems and saved as a CSV file can also be loaded. C) Pynapple offers various built-in loaders for commonly used data formats, as well as a template to easily design a customizable loader to adapt to any other format or specific task design.

data loading and synchronizing functions already included in the package for supported data types is a good starting point for any user writing a custom loading function (details this process are provided later).

While data types are usually specific to a recording modality (i.e., calcium imaging and electrophysiology), there are several pieces of metadata that are common to many experiments, such as the strain of the animal, age, sex, and name of the experimenter. When loading a session for the first time, the I/O process starts with a graphical user interface (GUI) in which the user can quickly and easily input the general information as well as any session epoch and behavioral tracking data (**Fig. 3b** [↗](#)). This information is saved in a *BaseLoader* class.

General session information is common across experimental sessions, however specialized data streams are usually specific to recording modalities. To cover the variety of preprocessing analysis pipelines currently used in systems neuroscience, the Pynapple I/O can load data formats from popular preprocessing pipelines (e.g., CNMF-E, Phy, NeuroSuite or Suite2P). This is implemented via a set of specialized object subclasses of the *BaseLoader* class, avoiding the need to redefine I/O operations in each subclass. This is a core aspect of object-oriented programming, and it means that these specialized I/O classes have all the methods and properties of the parent *BaseLoader* objects. This ensures compatibility across various loading functions. However, once generated, these specialized I/O classes are unique and independent from each other, ensuring long-term backward compatibility. For instance, if the spike sorting tool Phy changes its output in the future, this would not affect the “Neurosuite” IO class as they are independent of each other. This allows each tool to be updated or modified independently, without requiring changes to the other tool or the overall data format.

Like the *BaseLoader* class, a specialized GUI for electrophysiology and calcium imaging is provided, with relevant metadata fields, for example electrode position in electrophysiology and type of fluorescence indicator in calcium imaging (**Fig. 3B** [↗](#)).

To avoid repeating the process of inputting session information and synchronization of multiple data streams, Pynapple saves all synchronized data into a unique file and can accommodate a wide range of neuroscientific data types. Recently, Neurodata Without Borders (NWB) [26](#)[↗](#),[27](#)[↗](#) has emerged as a flexible data format used for public data sharing and large databases such as those collected by the Allen Institute. Thus, we chose to use the NWB format for fast and universal data loading and saving with Pynapple. The *BaseLoader* is responsible for initializing the NWB file within the session folder (i.e. it creates a new NWB file if none is present) (**Fig. 3C** [↗](#)). Converting user’s data to NWB format encourages standardization and can facilitate sharing both data and analysis pipelines written with Pynapple.

Many other preprocessing pipelines exist and can often be unique to a lab or even to an individual project. To accommodate present and future needs for these specific pipelines, the documentation of Pynapple provides an easy-to-follow recipe for creating a custom I/O class that inherits the *BaseLoader* and can interact with a pre-existing NWB file. There are multiple benefits of the inheritance approach of data loading classes within the I/O layer of Pynapple. First, future development of new I/O classes will not affect the core and processing layers of Pynapple. This ensures long-term stability of the package. Second, users can develop their own custom I/O using available template classes. Pynapple already includes several of such templates and we expect this collection to grow in the future. Third, users can still use Pynapple without using the I/O layer of Pynapple. Last, in order to apply previous analyses, or analyses developed in another lab, to new data or data types a user only needs to develop a new I/O class for their data. This will import the data to the common Pynapple core from which the same analysis pipeline can be used.

Foundational data processing

The basic methods that manipulate the core objects in Pynapple allow users to perform common, but powerful, neuroscience analyses (**Fig. 2** [↗](#)). These analyses are easy to use because they describe the relationships between time series objects, while requiring the fewest number of parameters to be set by the user. This minimizes complexity, while maximizing generalizability. The operations in Pynapple can recreate neuroscience analyses from a broad number of subdisciplines. These analyses form the foundation of neuroscience data analysis in Pynapple. To illustrate the versatility of Pynapple and how it can be used, we reanalyzed five openly available datasets.

The first foundational analysis is computing neural tuning curves. Tuning curves relate specific stimuli to the firing rate of neurons. To this end, Pynapple computes the firing rate of a neuron (or any other timestamped data) during each epoch in an *IntervalSet* object, for example for discrete conditions such as “ON/OFF” stimuli. Tuning curves can also be computed with respect to a continuous feature. Once computed, Pynapple is able to use tuning curves from a population of neurons to decode stimuli using a Bayesian decoder ^{32,33} [↗](#) (**Fig. 4A** [↗](#)).

The second foundational analysis is computing auto- and cross-correlograms of event data. In the most abstract sense, these correlograms show the relationship between previous and future events and a current event at time 0. In Pynapple, cross-correlograms can be generated for any two series of events by computing the event rate for each time bin of a target time series relative to each event of a reference time series. Commonly, this is used to examine the likelihood of an action potential in a neuron relating to a previous or future action potential in the same neuron (auto-correlogram) or in another neuron (cross-correlogram) (**Fig. 4B** [↗](#)). However, Pynapple does not limit this function to spiking data and correlograms may be performed on any event-based data.

The third and final foundational analysis is peri-event alignment. This involves aligning a specified window from *Ts/Tsd/TsGroup* data to a specific *Ts*, known as “TimeStamp Reference”. This allows users to align data to specific points in time, and measure changes in rates around this specified time point (**Fig. 4C** [↗](#)). One example where this function is useful is aligning neuronal spikes to specific stimuli, such as optogenetic illumination, presentation of a tone, or electrical stimulation.

Some of the analyses presented so far are designed for spikes (and discrete events in general) and cannot be applied for continuous traces such as calcium imaging data. Pynapple includes specialized functions that can compute the tuning of a continuous value with respect to a feature, as shown for the modulation of fluorescence in calcium imaging with respect to the speed of the animal (**Fig. 5A** [↗](#)) or of the position of a vertical bar on a screen in the fly’s ellipsoid body (**Fig. 5B** [↗](#)).

The examples shown in **Figures 4** [↗](#) and **5** [↗](#) show how these core analyses are useful for rapid data screening with just a few lines of code in a Jupyter notebook, for example. Overall, these foundational functions form the building blocks of most other analyses in systems neuroscience.

Importantly, they are for the most part built-in and only depend on a few widely used external packages. This ensures that the package can be used in a near stand-alone fashion, without relying on packages that are at risk of not being maintained or of not being compatible in the near future. All other developments of analysis pipelines take place outside Pynapple, ensuring the core package is only updated rarely and remains lightweight.

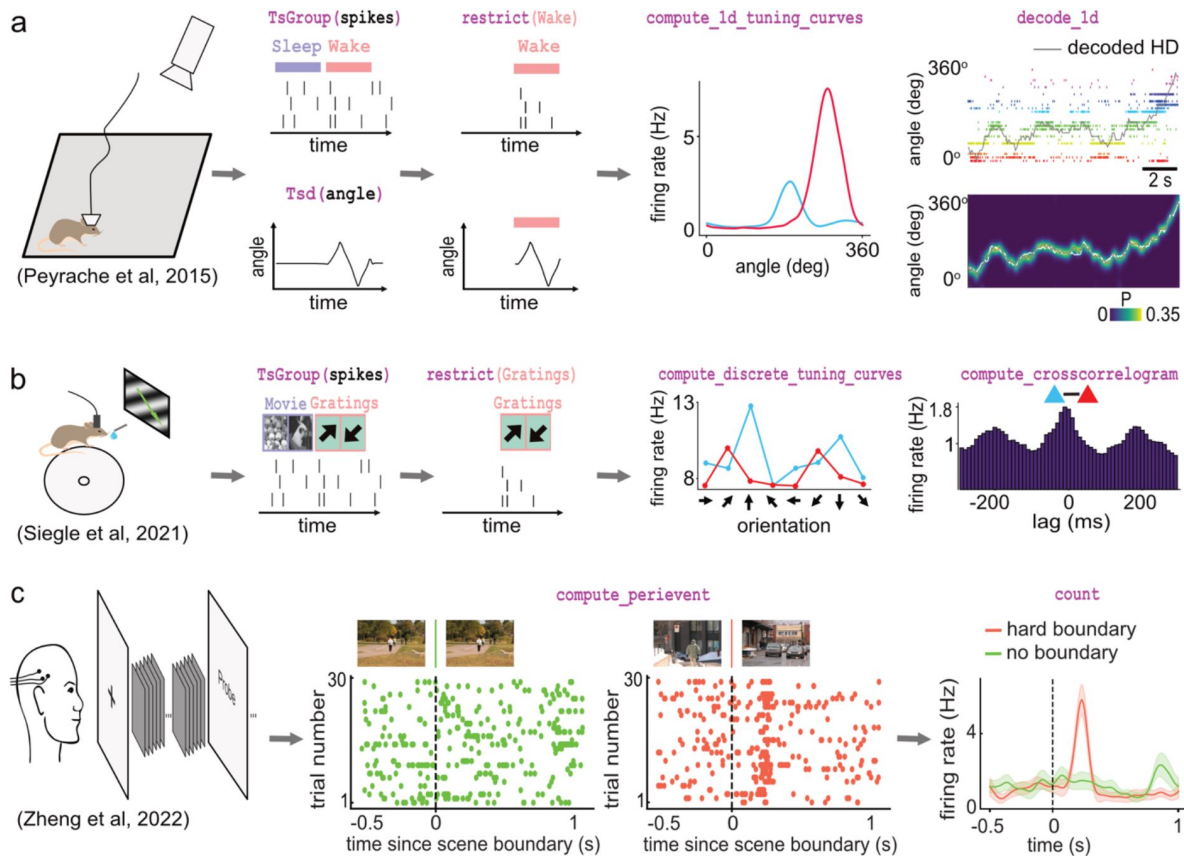


Fig. 4

Examples of foundational analysis across various electrophysiological datasets using Pynapple.

A) Analysis of an ensemble of head-direction cells. From left to right: data were collected in a freely moving mouse randomly foraging for food; all data are restricted to the wake epoch (i.e., during exploration); the tuning curve of two neurons relative to the animal's head-direction; animal's head-direction is decoded from the neuronal ensemble. Data from refs. [34](#), [35](#). B) Analysis of V1 neurons during visual stimulation. From left to right: the mouse was recorded while being head-fixed and presented with drifting gratings; spikes, stimulation and epochs are shown; example tuning curves of two V1 neurons, showing their firing rates for different grating orientations; example cross-correlation between two V1 neurons, showing an oscillatory co-modulation at about 5Hz during visual stimulation. Data from ref. [36](#). C) Analysis of medial temporal lobe neurons in human epileptic subjects. From left to right: subjects, implanted with hybrid deep electrodes, were shown a series of short clips; raster plot of a single neuron around continuous movie shot trials (green) and hard boundary trials, which are transitions between two unrelated movies (orange); peri-event neuronal firing rate for both trial types. Data from ref. [37](#). Images in panels b and c are from ref. [38](#). The analysis code used to generate this figure can be found on the Pynapple Organization Github repository: <https://github.com/pynapple-org/pynapple-paper-2023>

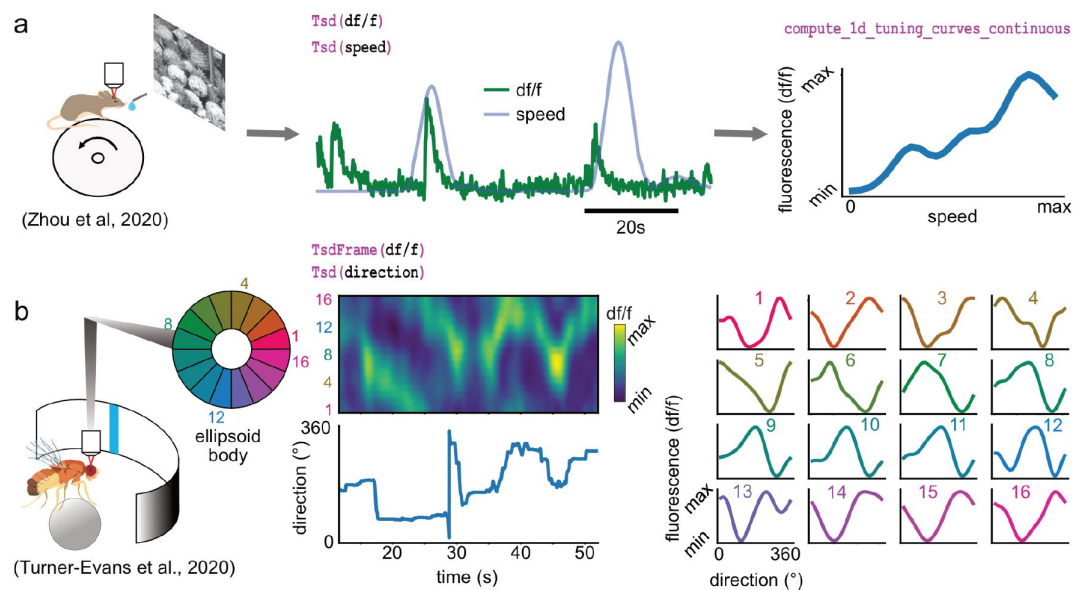


Fig. 5

Examples of foundational analysis across various calcium imaging datasets using Pynapple.

A) Analysis of a V1 neuron during visual stimulation. From left to right: the mouse was recorded while being head-fixed on a running wheel and presented with natural scene movies; fluorescence traces from a pre-processed region of interest and running speed are loaded; continuous tuning curve is directly obtained from fluorescence and speed. Data from ref. [39](#). Image is from ref. [38](#). B) Analysis of neuronal activity in the fly central complex. From left to right: a *Drosophila melanogaster* is tethered to a calcium imaging setup while the position of a vertical bar is in closed loop with the fly's movements on a ball; calcium activity in the ellipsoid body is divided into 16 wedges; example fluorescence trace and direction of the fly. Tuning curves are obtained as in A, with the direction as feature. Data from ref. [40](#).

Pynacollada: a collaborative library for specialized and continuously updated data analyses

Pynapple is designed to be stable in the foreseeable future and its core functionality is not meant to be modified. However, actual data analysis usually requires more than the available core functions. This type of data analysis is “fluid”, constantly updated by new software developments and theoretical work. Furthermore, this kind of development is collaborative in nature and the supervision of such projects is less sensitive than that of a stable package. To balance the needs for stability and flexibility, high-level functions were separated from Pynapple and included instead in Pynacollada: the Pynapple Collaborative repository hosted on GitHub.

Complex analyses are added to Pynacollada in the form of libraries. Each library developed for Pynacollada takes the form of a Jupyter notebook (or python scripts) which guides the user through the analysis step-by-step. As such, libraries built for Pynacollada should provide training, promote good practice in programming, and allow users to easily adapt code to their own project. Examples of complex analyses currently handled by Pynacollada are outlined below (Fig. 6 [↗](#)).

Recent advances in the application of manifold theory to neural data analysis have allowed neuroscientists to project high-dimensional data into three or fewer dimensions ^{41 [↗](#)–43 [↗](#)}. The structure of these projections reflects the structure of these higher dimensional processes, allowing us to infer the information encoded by the population. The Pynacollada “neural_manifold” library contains a Jupyter notebook that provides a step-by-step process for recreating a ring manifold using spiking data recorded from a population of head-direction neurons (Fig. 6 [↗](#)). This code can be adapted by the end-user for analysis of their own data by simply importing their own data and refactoring the parameters to suit their needs.

A second complex analysis handled by Pynacollada is sharp wave-ripple (SWR) detection. Detecting oscillatory events is a routine procedure in electrophysiology, yet usually depends on many arbitrary choices of parameters. In this case, the Jupyter notebook showcases an example of detecting SWRs, a well-characterized oscillation of the hippocampus (Fig. 6 [↗](#)).

In addition, Pynacollada currently includes libraries for spike waveform processing, EEG analysis, and video tracking, among others. We invite the community to contribute to this repository by improving current libraries or upload new ones. For new libraries, only rapid screening and tests will be performed, but the code will not go through the kind of validation that is in place for Pynapple as an external library will never affect the functioning of the core package. The documentation describes what is expected in each library to simplify readability, sharing, and maintenance and, overall, how libraries should conform to Pynacollada standards. We hope this will be broadly adopted by the community, allowing researchers across labs to easily share their code.

Discussion

Here we introduced Pynapple, a lightweight and open-source python toolbox for neural data analysis. The goal of this package is to offer a versatile set of tools to study typical neurophysiological and behavioral data, specifically time series (e.g., spike times, behavioral events, and continuous time series) and time intervals (e.g., trials and brain states). It also provides users with generic functions for neuroscience analyses such as tuning curves and cross-correlograms. Finally, Pynapple was designed to rely on a minimum number of dependencies, which are themselves very common and thus highly stable. As such, accessibility is the guiding axiom of Pynapple.

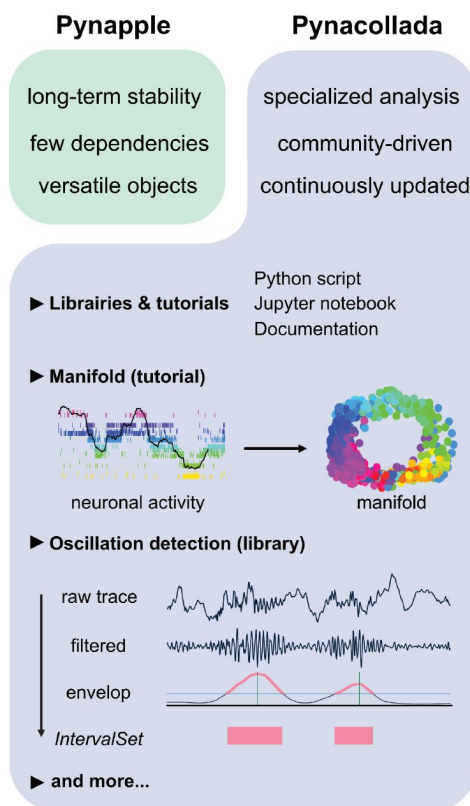


Fig. 6

The Pynapple collaborative data analysis repository (Pynacollada) environment.

Unlike Pynapple, which is designed for long-term stability, Pynacollada is a repository of project-oriented libraries. This way, the community can collaborate on constantly evolving data analysis code without affecting the functionality of the core pipeline. Each project should include a script that can be called for specific functions and/or Jupyter notebooks to showcase the use of the code, as well as proper documentation. Pynacollada already includes several libraries and/or tutorials, including but not limited to: (1) a tutorial on manifold analysis, covering how to project neuronal data on low-dimensional subspace using various machine learning techniques; (2) a library for oscillation detection in local field potentials, which takes raw broadband traces as inputs and outputs *IntervalSet* objects corresponding to the start and end times of oscillation bouts.

The path from data collection to reliable results involves a number of critical steps: exploratory data analysis, development of an analysis pipeline that can involve custom-made developed processing steps, and ideally the use of that pipeline and others to replicate the results. Pynapple provides a platform for these steps.

The design of Pynapple is centered around the manipulation of simple, abstract objects that are common to most neurophysiological and behavioral datasets. The core of Pynapple is built around five objects: Timestamps (*Ts*) and group of Timestamps (*TsGroup*), Time Series Data (*Tsd*) and ensemble of co-registered Tsd (*TsdFrame*), as well as *IntervalSets*. These objects can be manipulated with properties that are, in most cases, common to all objects. Building around these fundamental objects and properties means Pynapple is highly flexible and able to handle most neurophysiological and behavioral datasets, making it accessible to most systems neuroscientists.

Pynapple was developed to be lightweight, stable, and simple. As simplicity does not necessarily imply backward compatibility (i.e. long-term stability of the code), Pynapple main objects and their properties will remain the same for the foreseeable future, even if the code in the backend may eventually change (e.g. not relying on Pandas in future version). The small number of external dependencies also decrease the need to adapt the code to new versions of external packages. This approach favors long-term backward compatibility.

Data in neuroscience vary widely in their structure, size, and need for pre-processing. Pynapple is built around the idea that raw data have already been pre-processed (for example, spike sorting and detection of ROIs). According to the FAIR principles, pre-processed data should interoperate across different analysis pipelines. Pynapple makes this interoperability possible as, once the data are loaded in the Pynapple framework, the same code can be used to analyze different datasets. Specifically, to simplify analysis for users, Pynapple offers simple wrappers for loading data with popular preprocessing pipelines. However, to be fully accessible, it is not sufficient for a package's core operations to be able to process all data types in theory. Data produced in neuroscience has a wide variety of file types, which are often only loaded by specific analysis software. Data is also largely experiment-specific. To unify these disparate file types and configurations, Pynapple's data loader is customizable. In addition to being able to load current popular data formats, this customizable data-loader means emerging file formats may continue to be loaded in the future, without significant overhauls to the main package. This offers Pynapple long-term stability and means that Pynapple will continue to remain accessible in the foreseeable future. To note, Pynapple can be used without the I/O layer and independent of NWB for generic, on-the-fly analysis of data.

In further pursuit of accessibility, from these simple objects and properties, Pynapple has several built-in, foundational analyses that are common across the field of systems neuroscience. These foundational analyses include computing neural tuning curves, computing auto-/cross-correlograms, peri-event alignment, and performing Bayesian decoding. From these foundational analyses, higher order analyses can be developed. However, these higher order analyses are more prone to customization, thereby making them relatively more flexible. As such, higher order analyses are stored in the collaborative repository known as Pynacollada. This keeps the core Pynapple package stable, while allowing the user to integrate new advances in neurophysiological and behavioral analysis into their workflow.

Other software provide programming environments which deal with common neuroscientific data and an interface between stored data and analytical methods¹⁰[\[10\]](#). However, one problem that arises from this structure is that objects and data structures are rigidly defined, leading to a lack of versatility for new types of data or task design. In contrast, Pynapple offers a more flexible working environment and will remain accessible even as user requirements change.

While Pynapple expands accessibility to data analysis, it has some limitations inherent to its design. The first issue is that Pynapple is currently only available through Python. Thus, some transition is required for those primarily trained in other programming languages commonly used in neuroscience, including MATLAB and Julia. The design of the package around objects is a strength in many regards but could represent a challenge for users who are not accustomed to this programming approach. We have addressed this concern by providing users with detailed documentation, which includes a broad variety of examples. We will also keep on providing training opportunities for all future users. Last, Python code may run slower than similar code written in other languages. Pynapple is based on Pandas, whose methods are already highly optimized. Yet, current development is underway to improve computation speed and these developments are transparent for the users as they won't change the organization of the package.

Soon, Pynapple will be part of an entire suite of plugin libraries that we are developing to further enhance Pynapple. To keep Pynapple robust and stable, we will develop these plugins as standalone packages. These external packages will include an automated datalogger for recapitulating analyses, an on-line visualizer for Pynapple objects, and a package for parallel computing in Pynapple. This will address the speed issue inherent to code written in Python by allowing multiple analyses to be performed simultaneously. These packages will begin to address the limitations of Pynapple we described previously, enhancing the long-term stability of Pynapple, and streamlining accessibility for its users.

Data Availability

All data used in this manuscript are publicly available and were previously published.

Extracellular recordings from multi-site silicon probes in the anterior thalamus and subicular formation of freely moving mice.: Peyrache, A., Petersen, P. C. & Buzsaki, G., 2015, <http://dx.doi.org/10.6080/K0G15XS1> , CRCNS, SCR:006907;

Survey of spiking in the mouse visual system reveals functional hierarchy: Joshua H. Siegle, Xiaoxuan Jia, Séverine Durand, Sam Gale, Corbett Bennett, Nile Graddis, Gregory Heller, Tamina K. Ramirez, Hannah Choi, Jennifer A. Luviano, Peter A. Groblewski, Ruweida Ahmed, Anton Arkhipov, Amy Bernard, Yazan N. Billeh, Dillan Brown, Michael A. Buice, Nicolas Cain, Shiella Caldejon, Linzy Casal, Andrew Cho, Maggie Chvilicek, Timothy C. Cox, Kael Dai, Daniel J. Denman, Saskia E. J. de Vries, Roald Dietzman, Luke Esposito, Colin Farrell, David Feng, John Galbraith, Marina Garrett, Emily C. Gelfand, Nicole Hancock, Julie A. Harris, Robert Howard, Brian Hu, Ross Hytten, Ramakrishnan Iyer, Erika Jessett, Katelyn Johnson, India Kato, Justin Kiggins, Sophie Lambert, Jerome Lecoq, Peter Ledochowitsch, Jung Hoon Lee, Arielle Leon, Yang Li, Elizabeth Liang, Fuhui Long, Kyla Mace, Jose Melchior, Daniel Millman, Tyler Mollenkopf, Chelsea Nayan, Lydia Ng, Kiet Ngo, Thuyahn Nguyen, Philip R. Nicovich, Kat North, Gabriel Koch Ocker, Doug Ollerenshaw, Michael Oliver, Marius Pachitariu, Jed Perkins, Melissa Reding, David Reid, Miranda Robertson, Kara Ronellenfitch, Sam Seid, Cliff Slaughterbeck, Michelle Stoecklin, David Sullivan, Ben Sutton, Jackie Swapp, Carol Thompson, Kristen Turner, Wayne Wakeman, Jennifer D. Whitesell, Derric Williams, Ali Williford, Rob Young, Hongkui Zeng, Sarah Naylor, John W. Phillips, R. Clay Reid, Stefan Mihalas, Shawn R. Olsen & Christof Koch, 2021, <https://gui.dandiarchive.org/#/dandiset/000021> , Dandiset, 000021;

The Neuroanatomical Ultrastructure and Function of a Biological Ring Attractor: Daniel B. Turner-Evans, Kristopher T. Jensen, Saba Ali, Tyler Paterson, Arlo Sheridan, Robert P. Ray, Tanya Wolff, J. Scott Lauritzen, Gerald M. Rubin, Davi D. Bock, and Vivek Jayaraman, 2020, https://janelia.figshare.com/articles/dataset/OneColor_zip/12490373 , Allen, 12490373;

Neurons detect cognitive boundaries to structure episodic memories in humans: Zheng, Jie, Andrea GP Schjetnan, Mar Yebra, Bernard A. Gomes, Clayton P. Mosher, Suneil K. Kalia, Taufik A. Valiante, Adam N. Mamelak, Gabriel Kreiman, and Ueli Rutishauser., 2022, <https://doi.org/10.48324/dandi.000207/0.220216.0323>, Dandiset, 0.220216.0323;

EASE: EM-Assisted Source Extraction from calcium imaging data: Pengcheng Zhou, Jacob Reimer, Ding Zhou, Amol Pasarkar, Ian Kinsella, Emmanouil Froudarakis, Dimitri V Yatsenko, Paul G Fahey, Agnes Bodor, JoAnn Buchanan, Dan Bumbarger, Gayathri Mahalingam, Russel Torres, Sven Dorkenwald, Dodam Ih, Kisuk Lee, Ran Lu, Thomas Macrina, Jingpeng Wu, Nuno da Costa, R. Clay Reid, Andreas S Tolias, Liam Paninski, 2020, <https://www.microns-explorer.org/cortical-mm3#f-data>, Allen, cortical-mm3#f-data

Code Availability

All code is publicly available online under GPL-3.0 license.

Pynapple: <https://github.com/pynapple-org/pynapple>

Pynacollada: <https://github.com/PeyracheLab/pynacollada>

Code to generate **Figures 4** and **5**: <https://github.com/pynapple-org/pynapple-paper-2023>

Acknowledgements

This work was supported by a Canadian Research Chair in Systems Neuroscience, CIHR Project Grant 155957 and 180330, NSERC Discovery Grant RGPIN-2018-04600, the Canada-Israel Health Research Initiative, jointly funded by the Canadian Institutes of Health Research, the Israel Science Foundation, the International Development Research Centre, Canada and the Azrieli Foundation 108877-001, and the Tanenbaum Open Science Institute (AP).

Competing Interests

No competing interests disclosed.

References

1. Folk M., Heber G., Koziol Q., Pourmal E., Robinson D. (2011) **An overview of the HDF5 technology suite and its applications** in *Proceedings of the EDBT/ICDT 2011 Workshop on Array Databases* :36–47 <https://doi.org/10.1145/1966895.1966900>
2. Wells D. C., Greisen E. W. (1979) **Wells, D. C. & Greisen, E. W. FITS - a Flexible Image Transport System. 445 (1979). FITS - a Flexible Image Transport System 445**
3. Pedregosa F., et al. (2011) **Scikit-learn: Machine learning in Python** *J. Mach. Learn. Res* **12**:2825–2830
4. van der Maaten L. J. P., Hinton G. E. (2008) **Visualizing High-Dimensional Data Using t-SNE** *J. Mach. Learn. Res* **9**:2579–2605
5. McKinney W. (2011) **pandas: a foundational Python library for data analysis and statistics** *Python High Perform. Sci. Comput* **14**:1–9
6. Abraham A., et al. (2014) **Machine learning for neuroimaging with scikit-learn** *Front. Neuroinformatics* **14**
7. Tadel F., Baillet S., Mosher J. C., Pantazis D., Leahy R. M. (2011) **Brainstorm: A User-Friendly Application for MEG/EEG Analysis** *Comput. Intell. Neurosci* **2011**
8. Oostenveld R., Fries P., Maris E., Schoffelen J.-M. (2010) **FieldTrip: Open Source Software for Advanced Analysis of MEG, EEG, and Invasive Electrophysiological Data** *Comput. Intell. Neurosci* **2011**
9. Bokil H., Andrews P., Kulkarni J. E., Mehta S., Mitra P. P. (2010) **Chronux: A platform for analyzing neural signals** *J. Neurosci. Methods* **192**:146–151
10. Garcia S., et al. (2014) **Neo: an object model for handling electrophysiology data in multiple formats** *Front. Neuroinformatics* **8**
11. Freeman J., et al. (2014) **Mapping brain activity at scale with cluster computing** *Nat. Methods* **11**:941–950
12. Stevenson I. H., Kording K. P. (2011) **How advances in neural recording affect data analysis** *Nat. Neurosci* **14**:139–142
13. Urai A. E., Doiron B., Leifer A. M., Churchland A. K. (2022) **Large-scale neural recordings call for new insights to link brain and behavior** *Nat. Neurosci* **25**:11–19
14. Pachitariu M., Steinmetz N., Kadir S., Carandini M., Harris K. D. (2016) **Kilosort: realtime spike-sorting for extracellular electrophysiology with hundreds of channels** *bioRxiv* <https://doi.org/10.1101/061481>
15. Pachitariu M., et al. (2017) **Pachitariu, M. et al. Suite2p: beyond 10,000 neurons with standard two-photon microscopy. 061507 Preprint at 10.1101/061507 (2017).** <https://doi.org/10.1101/061507>

16. Hazan L., Zugaro M., Buzsáki G. (2006) **Klusters, NeuroScope, NDManager: A free software suite for neurophysiological data processing and visualization** *J. Neurosci. Methods* **155**:207–216
17. Fee M. S., Mitra P. P., Kleinfeld D. (1996) **Automatic sorting of multiple unit neuronal signals in the presence of anisotropic and non-Gaussian variability** *J. Neurosci. Methods* **69**:175–188
18. Harris K. D., Henze D. A., Csicsvari J., Hirase H., Buzsáki G. (2000) **Accuracy of tetrode spike separation as determined by simultaneous intracellular and extracellular measurements** *J. Neurophysiol* **84**:401–414
19. Yger P., et al. (2018) **A spike sorting toolbox for up to thousands of electrodes validated with ground truth recordings in vitro and in vivo** *eLife* **7**
20. Mathis A., et al. (2018) **DeepLabCut: markerless pose estimation of user-defined body parts with deep learning** *Nat. Neurosci* **21**:1281–1289
21. Zhou P., et al. (2018) **Efficient and accurate extraction of in vivo calcium signals from microendoscopic video data** *eLife* **7**
22. Mukamel E. A., Nimmerjahn A., Schnitzer M. J. (2009) **Automated Analysis of Cellular Signals from Large-Scale Calcium Imaging Data** *Neuron* **63**:747–760
23. Romano S. A., et al. (2017) **An integrated calcium imaging processing toolbox for the analysis of neuronal population dynamics** *PLOS Comput. Biol* **13**
24. Kaifosh P., Zaremba J. D., Danielson N. B., Losonczy A. (2014) **SIMA: Python software for analysis of dynamic fluorescence imaging data** *Front. Neuroinformatics* **8**
25. Pnevmatikakis E. A., Giovannucci A. (2017) **NoRMCorre: An online algorithm for piecewise rigid motion correction of calcium imaging data** *J. Neurosci. Methods* **291**:83–94
26. Teeters J. L., et al. (2015) **Neurodata Without Borders: Creating a Common Data Format for Neurophysiology** *Neuron* **88**:629–634
27. Rübél O., et al. (2022) **The Neurodata Without Borders ecosystem for neurophysiological data science** *eLife* **11**
28. Nasiotis K., et al. (2019) **Integrated open-source software for multiscale electrophysiology** *Sci. Data* **6**
29. Zugaro M. **Zugaro, M. Freely Moving Animal (FMA) toolbox.**
30. Ackermann E., Chu J., Dutta S., Kemere C. (2018) **Nelpy: Neuroelectrophysiology Object Model and Data Analysis in Python** *Nelpy: Neuroelectrophysiology Object Model and Data Analysis in Python*
31. Unakafova V. A., Gail A. (2019) **Comparing Open-Source Toolboxes for Processing and Analysis of Spike and Local Field Potentials Data** *Front. Neuroinformatics* **13**
32. Zhang K., Ginzburg I., McNaughton B. L., Sejnowski T. J. (1998) **Interpreting Neuronal Population Activity by Reconstruction: Unified Framework With Application to Hippocampal Place Cells** *J. Neurophysiol* **79**:1017–1044

33. Brown E. N., Frank L. M., Tang D., Quirk M. C., Wilson M. A. (1998) **A statistical paradigm for neural spike train decoding applied to position prediction from ensemble firing patterns of rat hippocampal place cells** *J. Neurosci* **18**
34. Peyrache A., Lacroix M. M., Petersen P. C., Buzsáki G. (2015) **Internally organized mechanisms of the head direction sense** *Nat. Neurosci* **18**:569–575
35. Peyrache A., Petersen P. C., Buzsáki G. **Extracellular recordings from multi-site silicon probes in the anterior thalamus and subicular formation of freely moving mice** <https://doi.org/10.6080/K0G15XS1>
36. Siegle J. H., et al. (2021) **Survey of spiking in the mouse visual system reveals functional hierarchy** *Nature* **592**:86–92
37. Zheng J., et al. (2022) **Neurons detect cognitive boundaries to structure episodic memories in humans** *Nat. Neurosci* **25**:358–368
38. Olmos A., Kingdom F. A. A. (2004) **A Biologically Inspired Algorithm for the Recovery of Shading and Reflectance Images** *Perception* **33**:1463–1473
39. Zhou P., et al. (2020) **EASE: EM-Assisted Source Extraction from calcium imaging data** *BioRxiv*
40. Turner-Evans D. B., et al. (2020) **The Neuroanatomical Ultrastructure and Function of a Biological Ring Attractor** *Neuron* **108**:145–163
41. Chaudhuri R., Gerçek B., Pandey B., Peyrache A., Fiete I. (2019) **The intrinsic attractor manifold and population dynamics of a canonical cognitive circuit across waking and sleep** *Nat. Neurosci* **22**:1512–1520
42. Viejo G., Peyrache A. (2020) **Precise coupling of the thalamic head-direction system to hippocampal ripples** *Nat. Commun* **11**
43. Gardner R. J., et al. (2022) **Toroidal topology of population activity in grid cells** *Nature* **602**:123–128

Article and author information

Guillaume Viejo

Montreal Neurological Institute and Hospital, McGill University, Montreal, QC, Canada

Daniel Levenstein

Montreal Neurological Institute and Hospital, McGill University, Montreal, QC, Canada, MILA – Quebec IA Institute

Sofia Skromne Carrasco

Montreal Neurological Institute and Hospital, McGill University, Montreal, QC, Canada

Dhruv Mehrotra

Montreal Neurological Institute and Hospital, McGill University, Montreal, QC, Canada
ORCID iD: [0000-0001-5813-3218](https://orcid.org/0000-0001-5813-3218)

Sara Mahallati

Montreal Neurological Institute and Hospital, McGill University, Montreal, QC, Canada

Gilberto R Vite

Montreal Neurological Institute and Hospital, McGill University, Montreal, QC, Canada

Henry Denny

Montreal Neurological Institute and Hospital, McGill University, Montreal, QC, Canada

Lucas Sjulson

Departments of Psychiatry and Neuroscience, Albert Einstein College of Medicine, Bronx, NY

Francesco P Battaglia

Donders Institute for Brain, Cognition and Behaviour, Radboud University, 6525AJ Nijmegen, The Netherlands

Adrien Peyrache

Montreal Neurological Institute and Hospital, McGill University, Montreal, QC, Canada

For correspondence: adrien.peyrache@mcgill.ca

Copyright

© 2023, Viejo et al.

This article is distributed under the terms of the [Creative Commons Attribution License](#), which permits unrestricted use and redistribution provided that the original author and source are credited.

Editors

Reviewing Editor

Caleb Kemere

Rice University, Houston, United States of America

Senior Editor

Timothy Behrens

University of Oxford, Oxford, United Kingdom

Reviewer #1 (Public Review):

A typical path from preprocessed data to findings in systems neuroscience often includes set of analyses that often share common components. For example, an investigator might want to generate plots that relating one time series (e.g., a set of spike times) to another (measurements of a behavioral parameter such as pupil diameter or running speed). In most cases, each individual scientist writes their own code to carry out these analyses, and thus the same basic analysis is coded repeatedly. This is problematic for several reasons, including the inefficiency of different people writing the same code over and over again.

This paper presents Pynapple, a python package that aims to address those problems.

Strengths:

The authors have identified a key need in the community - well written analysis routines that carry out a core set of functions and can import data from multiple formats. In addition, they recognized that there are some common elements of many analyses, particularly those involving timeseries, and their object-oriented architecture takes advantage of those commonalities to simplify the overall analysis process.

The package is separated into a core set of applications and another with more advanced applications, with the goal of both providing a streamlined base for analyses and allowing for implementations/inclusion of more experimental approaches.

Weaknesses:

The revised version of the paper does a very good job of addressing previous concerns. It would be slightly more accurate in the Highlights section to say "A lightweight and standalone package facilitating long-term backward compatibility" but this is a very minor issue.

- <https://doi.org/10.7554/eLife.85786.2.sa1>

Reviewer #2 (Public Review):

The manuscript by G. Viejo et al. describes a new open-source toolbox called Pynapple, for data analysis of electrophysiological recordings, calcium imaging, and behavioral data. It is an object-oriented python package, consisting of 5 main object types: timestamps (Ts), timestamped data (Tsd), TsGroup, TsdFrame, and IntervalSet. Each object has a set of built-in core methods and import tools for common data acquisition systems and pipelines.

Pynapple is a low-level package that uses NWB as a file format, and further allows for other more advanced toolsets to build upon it. One of these is called Pynacollada which is a toolset for data analysis of electrophysiological, imaging, and behavioral data.

Pynapple and Pynacollada have the potential to become very valuable and foundational tools for the analysis of neurophysiological data. NWB still has a steep learning curve and Pynapple offers a user-friendly toolset that can also serve as a wrapper for NWB.

- <https://doi.org/10.7554/eLife.85786.2.sa0>

Author Response

The following is the authors' response to the original reviews.

We would first like to thank the reviewers and the editor for their insightful comments and suggestions. We are particularly glad to read that our software package constitutes a set of "well-written analysis routines" which have "the potential to become very valuable and foundational tools for the analysis of neurophysiological data". We have updated the manuscript to address their remarks where appropriate.

Additionally, we would like to stress that this kind of tools is in continual development. As such, the manuscript offered a snapshot of the package at one point during this process, which in this case was several months ago at initial submission. Since then, several improvements were implemented. The manuscript has been further updated to reflect these more recent changes.

From the Reviewing Editor:

The reviewers identified a number of fundamental weaknesses in the paper.

1. For a paper demonstrating a toolbox, it seems that some example analyses showing the value of the approach (and potentially the advantage in simplification, etc over previous or other approaches) are really important to demonstrate.

As noted by the first reviewer, the online repository (i.e. GitHub page) conveys a better sense of the toolboxes' contribution to the field than the present manuscript. This is a fair remark but at the same time, it is unclear how to illustrate this in a journal article without dedicating a great deal of page space to presenting raw code, while online tools offer an easier and clearer way to do this. As a work-around, our strategy was to illustrate some examples of data analysis in Figures 4&5 by comparing each illustrated processing step to the corresponding command line used by the Pynapple package. Each step requires a single line of code, meaning that one only needs to write three lines of code to decode a feature from population activity using a Bayesian decoder (Fig. 4a), compute a cross-correlograms of two neurons during specific stimulus presentation (Fig. 4b) or compute the average firing rate of two neurons around a specific time of the experimental task (Fig. 4c). We believe that these visual aides make it unnecessary to add code in the main text of this manuscript. However, to aid reader understanding, we now provide clear references to online Jupyter notebooks which show how each figure was generated in figure legends as well as in the "Code Availability" section.

<https://github.com/pynapple-org/pynapple-paper-2023>

Furthermore, we have opted-in for the "Executable Research Articles" feature at eLife, which will make it possible to include live scripts and figures in the manuscript once it is accepted for publication. We do not know at this stage what it entails exactly, but we hope that Figures 4&5 will become live with this feature. The readers will have the possibility to see and edit the code directly within the online version of the manuscript.

1. The manuscript's claims about not having dependencies seem confusing.

We agree that this claim was somewhat unfounded. There are virtually no Python packages that do not have dependencies. Our intention was to say that the package had no dependencies outside the most common ones, which are Numpy, Scipy, and Pandas. Too many packages in the field tend to have long list of dependencies making long-term back-compatibility quite challenging. By keeping dependencies minimal, we hope to maximise the package's long term back-compatibility. We have rephrased this statement in the manuscript in the following sections:

Figure 1, legend.

"These methods depend only on a few, commonly used, external packages."

Section Foundational data processing: "they are for the most part built-in and only depend on a few widely-used external packages. This ensures that the package can be used in a near stand-alone fashion, without relying on packages that are at risk of not being maintained or of not being compatible in the near future."

1. Given its significant relevance, it seems important to cite the FMATool and describe connections between it (or analyses based on it) and the presented work.

Indeed, although we had already cited other toolboxes (including a review covering the topic comprehensively), we should have included this one in the original manuscript.

Unfortunately, to the best of our knowledge, this toolbox is not citable (there is no companion paper). We have added a reference to it in plain text.

1. Some discussion of integration between Pynapple and the rest of a full experimental data pipeline should be discussed with regard to reproducibility.

This is an interesting point, and the third paragraph of the discussion somewhat broached this issue. Pynapple was not originally designed to pre-process data. However, it can, in theory, load any type of data streams after the necessary pre-processing steps. Overall, modularity is a key aspect of the Pynapple framework, and this is also the case for the integration with data pre-processing pipelines, for example spike sorting in electrophysiology and detection of region of interest in calcium imaging. We do not think there should be an integrated solution to the problem but, instead, to make it possible that any piece of code can be used for data irrespective of their origin. This is why we focused on making data loading straightforward and easy to adapt to any particular situation. To expand on this point and make it clear that Pynapple is not meant to pre-process data but can, in theory, load any type of data streams after the necessary pre-processing steps, we have added the following sentences to the aforementioned paragraph:

“Data in neuroscience vary widely in their structure, size, and need for pre-processing. Pynapple is built around the idea that raw data have already been pre-processed (for example, spike sorting and detection of ROIs).”

1. Relatedly, a description of how data are stored after processing (i.e., how precisely are processed data stored in NWB format).

We agree that this is a critical issue. NWB is not necessarily the best option as it is not possible to overwrite in a NWB file. This would require the creation of a new NWB file each time, which is computationally expensive and time consuming. It also further increases the odds of writing error. Theoretically, users who need to store intermediate results in a flexible way could use any methods they prefer, writing their own data files and wrappers to reload these data into Pynapple objects. Indeed, it is not easy to properly store data in an object-specific manner. This is a long-standing issue and one we are currently working to resolve.

To do so, we are developing I/O methods for each Pynapple core objects. We aim to provide an output format that is simple to read and backward compatible in future Pynapple releases. This feature will be available in the coming weeks. To note, while NWB may not be the central data format of Pynapple in future releases, it has become a central node in the neuroscience ecosystem of software. Therefore, we aim to facilitate the interaction of users with reading and writing for this format by developing a set of simple standalone functions.

Reviewer #1 (Public Review):

A typical path from preprocessed data to findings in systems neuroscience often includes a set of analyses that often share common components. For example, an investigator might want to generate plots that relate one time series (e.g., a set of spike times) to another (measurements of a behavioral parameter such as pupil diameter or running speed). In most cases, each individual scientist writes their own code to carry out these analyses, and thus the same basic analysis is coded repeatedly. This is problematic for several reasons, including the waste of time, the potential for errors, and the greater difficulty inherent in sharing highly customized code.

This paper presents Pynapple, a python package that aims to address those problems.

Strengths:

The authors have identified a key need in the community - well-written analysis routines that carry out a core set of functions and can import data from multiple formats. In addition, they recognized that there are some common elements of many analyses, particularly those involving timeseries, and their object-oriented architecture takes advantage of those commonalities to simplify the overall analysis process.

The package is separated into a core set of applications and another with more advanced applications, with the goal of both providing a streamlined base for analyses and allowing for implementations/inclusion of more experimental approaches.

Weaknesses:

There are two main weaknesses of the paper in its present form.

First, the claims relating to the value of the library in everyday use are not demonstrated clearly. There are no comparisons of, for example, the number of lines of code required to carry out a specific analysis with and without Pynapple or Pynacollada. Similarly, the paper does not give the reader a good sense of how analyses are carried out and how the object-oriented architecture provides a simplified user interaction experience. This contrasts with their GitHub page and associated notebooks which do a better job of showing the package in action.

As noted in the response to the Reviewing Editor and response to the reviewer's recommendation to the authors below, we have now included links to Jupyter notebooks that highlight how panels of Figures 4 and 5 were generated (<https://github.com/pynapple-org/pynapple-paper-2023>). However, we believe that including more code in the manuscript than what is currently shown (I.e. abbreviated call to methods on top of panels in Figs 4&5) would decrease the readability of the manuscript.

Second, the paper makes several claims about the values of object-oriented programming and the overall design strategy that are not entirely accurate. For example, object-oriented programming does not inherently reduce coding errors, although it can be part of good software engineering. Similarly, there is a claim that the design strategy "ensures stability" when it would be much more accurate to say that these strategies make it easier to maintain the stability of the code. And the authors state that the package has no dependencies, which is not true in the codebase. These and other claims are made without a clear definition of the properties that good scientific analysis software should have (e.g., stability, extensibility, testing infrastructure, etc.).

Following the reviewer's comment, we have rephrased and clarified these claims. We provide detailed response to these remarks in the recommendations to authors below.

There is also a minor issue - these packages address an important need for high-level analysis tools but do not provide associated tools for preprocessing (e.g., spike sorting) or for creating reproducible pipelines for these analyses. This is entirely reasonable, in that no one package can be expected to do everything, but a bit deeper account of the process that takes raw data and produces scientific results would be helpful. In addition, some discussion of how this package could be combined with other tools (e.g., DataJoint, Code Ocean) would help provide context for where Pynapple and Pynacollada could fit into a robust and reliable data analysis ecosystem.

We agree the better explaining how Pynapple is integrated within data preprocessing pipelines is essential. We have clarified this aspect in the manuscript and provide more details below.

Reviewer #1 (Recommendations For The Authors):

Page 1

- Title

The authors should note that the application name- "Pynapple" could be confused with something from Apple. Users may search for "Pyapple" as many python applications contain "py" like "Numpy". "Pyapple" indeed is a Python Apple that works with Apple products. They could consider "NeuroFrame", "NeuroSeries" or "NeuroPandas" to help users realize this is not an apple product.

We thank the referee for this interesting comment. However, we are not willing to make such change at this point. The community of users has been growing in the last year and it seems too late to change the name. To note, it is the first time such comment is made to us and it does not seem that users and collaborators are confused with any Apple products.

- Abstract

The authors mentioned that the Pynapple is "fully open source". It may be better to simply say it is "open source".

We agree, corrected.

Assuming the authors keep the name, it would be helpful if the full meaning of Pynapple - Python Neural Analysis Package was presented as early as possible.

Corrected in the abstract.

- Highlight

An application being lightweight and standalone does not imply nor ensure backward compatibility. In general, it would be useful if the authors identified a set of desirable code characteristics, defined them clearly in the introduction, and then describe their software in terms of those characteristics.

Thank you for your comment. We agree that being lightweight and standalone does not necessarily imply backward compatibility. Our intention was to emphasize that Pynapple is designed to be as simple and flexible as possible, with a focus on providing a consistent interface for users across different versions. However, we understand that this may not be enough to ensure long-term stability, which is why we are committed to regular updates and maintenance to ensure that the code remains functional as the underlying code base (Python versions, etc.) changes.

Regarding your suggestion to identify a set of desirable code characteristics, we believe this is an excellent idea. In the introduction, we briefly touch upon some of the core principles that guided our development of Pynapple: a lightweight, stable, and simple package. However, we acknowledge that providing a more detailed discussion of these characteristics and how they relate to the design of our software would be useful for readers. We have added this paragraph in the discussion:

"Pynapple was developed to be lightweight, stable, and simple. As simplicity does not necessarily imply backward compatibility (i.e. long-term stability of the code), Pynapple main

objects and their properties will remain the same for the foreseeable future, even if the code in the backend may eventually change (e.g. not relying on Pandas in future version). The small number of external dependencies also decrease the need to adapt the code to new versions of external packages. This approach favors long-term backward compatibility."

Page 2

- The authors wrote -

"Despite this rapid progress, data analysis often relies on custom-made, lab-specific code, which is susceptible to error and can be difficult to compare across research groups."

It would be helpful to add that custom-made, lab-specific code can lead to a violation of FAIR principles (https://en.wikipedia.org/wiki/FAIR_data). More generally, any package can have errors, so it would be helpful to explain any testing regiments or other approach the authors have taken to ensure that their code is error-free.

We understand the importance of the FAIR principles for data sharing. However, Pynapple was not designed to handle data through their pre-processing. The only aspect that is somehow covered by the FAIR principles is the interoperability, but again, it is a requirement for the data to interoperate with different storage and analysis pipelines, not of the analysis framework itself. Unlike custom-made code, Pynapple will make interoperability easier, as, in theory, once the required data loaders are available, any analysis could be run on any dataset. We have added the following sentence to the discussion:

"Data in neuroscience vary widely in their structure, size, and need for pre-processing. Pynapple is built around the idea that raw data has already been pre-processed (for example, spike sorting and ROI detection). According to the FAIR principles, pre-processed data should interoperate across different analysis pipelines. Pynapple makes this interoperability possible as, once the data are loaded in the Pynapple framework, the same code can be used to analyze different datasets"

- The authors wrote -

"While several toolboxes are available to perform neuronal data analysis (see ref. 29 for review), most of these programs focus on producing high-level analysis from specified types of data and do not offer the versatility required for rapidly-changing analytical methods and experimental methods."

Here it would be helpful if the authors could give a more specific example or explain why this is problematic enough to be a concern. Users may not see a problem with high-level analysis or using specific data types.

Again, we apologize for not fully elaborating upon our goals here. Our intention was to point out that toolboxes often focus on one particular case of high-level analysis. In many cases, such packages lack low level analysis features or the flexibility to derive new analysis pipelines quickly and effortlessly. Users can decide to use low-level packages such as Pandas, but in that case, the learning curve can be steep for users with low, if any, computational background. The simplicity of Pynapple, and the set of examples and notebooks, make it possible for individuals who start coding to be quickly able to analyze their data.

As we do not want to be too specific at this point of the manuscript (second paragraph of the intro) and as we have clarified many of the aspects of the toolbox in the new revised version, we have only added the following sentence to the paragraph:

“Users can decide to use low-level data manipulation packages such as Pandas, but in that case, the learning curve can be steep for users with low, if any, computational background.”

- *The authors wrote -*

"To meet these needs, a general toolbox for data analysis must be designed with a few principles in mind"

Toolboxes based on many different principles can solve problems. It is likely more accurate to say that the authors designed their toolbox with a particular set of principles in mind. A clear description of those principles (as mentioned in the comment above) would help the reader understand why the specific choices made are beneficial.

We agree that these are not “universal” principles and clearly more the principles we had in mind when we designed the package. We have clarified these principles and made clear that these are personal point of views.

We have rephrased the following paragraph:

“To meet these needs, we designed Pynapple, a general toolbox for data analysis in systems Neuroscience with a few principles in mind.“

- *The authors wrote -*

"The first property of such a toolbox is that it should be object-oriented, organizing software around data."

What facts make this true? For example, React is a web development library. A common approach to using this library is to use Hooks (essentially a collection of functions). This is becoming more popular than the previous approach of using Components (a collection of classes). This is an example of how Object-oriented programming is not always the best solution. In some cases, for example, object-oriented coding can cause problems (e.g. it can be hard to find the place where a given function is defined and to figure out which version is being used given complex inheritance structures.)

In general, key selling points of object-oriented programming are extension, inheritance, and encapsulation. If the authors want to retain this text (which would be entirely reasonable), it would be helpful if they explained clearly how an object-oriented approach enables these functions and why they are critical for this application in particular.

The referee makes a particularly important point. We are aware of the limits of OOP, especially when these objects become over-complex, and that the inheritance become unclear.

We have clarified our goal here. We believe that in our case, OOP is powerful and, overall, is less error-prone than a collection of functions. The reasons are the following:

An object-oriented approach facilitates better interactions between objects. By encapsulating data and behavior within objects, object-oriented programming promotes clear and well-defined interfaces between objects. This results in more structured and manageable code, as objects communicate with each other through these well-defined interfaces. Such improved interactions lead to increased code reliability.

Inheritance, a key concept in object-oriented programming, allows for the inheritance of properties. One important example of how inheritance is crucial in the Pynapple framework is the time support of Pynapple objects. It determines the valid epoch on which the object is defined. This property needs to be carried over during different manipulations of the object. Without OOP, this property could easily be forgotten, resulting in erroneous conclusions for many types of analysis. The simplest case is the average rate of a TS object: the rate must be computed on the time support (a property of TS objects), not the beginning to the end of the recording (or of a specific epoch, independent of the TS). Finally, it is easier to access and manipulate the meta information of a Pynapple object than without using objects.

- The authors wrote -

"drastically diminishing the odds of a coding error"

This seems a bit strong here. Perhaps "reducing the odds" would be more accurate.

We agree. Now changed.

Page 3

- The authors wrote -

". Another property of an efficient toolbox is that as much data as possible should be captured by only a small number of objects This ensures that the same code can be used for various datasets and eliminates the need of adapting the structure"

It may be better to write something like - "Objects have a collection of preset variables/values that are well suited for general use and are very flexible." Capturing "as much data as possible" may be confusing, because it's not the amount that this helps with but rather the variety.

We thank the referee for this remark. We have rephrased this sentence as follows:

"Another property of an efficient toolbox is that a small number of objects could virtually represents all possible data streams in neuroscience, instead of objects made for specific physiological processes (e.g. spike trains)."

- The authors wrote -

"The properties listed above ensure the long-term stability of a toolbox, a crucial aspect for maintaining the code repository. Toolboxes built around these principles will be maximally flexible and will have the most general application"

There are two issues with this statement. First, ensuring long-term stability is only possible with a long- term commitment of time and resources to ensure that that code remains functional as the underlying code base (python versions, etc.) changes. If that is something you are commising to, it would be great to make that clear. If not, these statements need to be less firm.

Second, it is not clear how these properties were arrived at in the first place. There are things like the FAIR Principles which could provide an organizing framework, ideally when combined with good so<ware engineering practices, and if some more systematic discussion of these properties and their justification could be added, it would help the field think about this issue more clearly.

The referee makes a valid point that ensuring long-term stability requires a long-term commitment of time and resources to maintain the code as the underlying technology evolves. While we cannot make guarantees about the future of Pynapple, we believe that one of the best ways to ensure long-term stability is by fostering a strong community of users and contributors who can provide ongoing support and development. By promoting open-source collaboration and encouraging community involvement, we hope to create a sustainable ecosystem around Pynapple that can adapt to changes in technology and scientific practices over time. Ultimately, the longevity of any scientific tool depends on its adoption and use by the research community, and we hope that Pynapple can provide value to neuroscience researchers and continue to evolve and improve as the field progresses.

It is noteworthy that the first author, and main developer of the package, has now been hired as a data scientist at the Center for Computational Neuroscience, Flatiron Institute, to explicitly continue the development of the tool and build a community of users and contributors.

- *The authors wrote -*

"each with a limited number of methods..."

This may give the impression that the functionality is limited, so rephrasing may be helpful.

Indeed! We have now rephrased this sentence:

"The core of Pynapple is five versatile timeseries objects, whose methods make it possible to intuitively manipulate and analyze the data."

- *The authors wrote that object-oriented coding*

"limits the chances of coding error"

This is not always the case, but if it is the case here, it would be helpful if the authors explain exactly how it helps to use object-oriented approaches for this package.

We agree with the referee that it is not always the case. As we explained above, we believe it is less error-prone that a collection of functions. Quite often, it also makes it easier to debug. We have changed this sentence with the following one:

"Because objects are designed to be self-contained and interact with each other through well-defined methods, users are less likely to make errors when using them. This is because objects can enforce their own internal consistency, reducing the chances of data inconsistencies or unexpected behavior. Overall, OOP is a powerful tool for managing complexity and reducing errors in scientific programming."

- *Fig 1*

In object-oriented programming, a class is a blueprint for the classes that inherit it. Instantiating that

class creates an object. An object contains any or all of these - data, methods, and events. The figure could be improved if it maintained these organizational principles as figure properties.

We agree with the referee's remark regarding the logic of objects instantiation but how this could be incorporated in Fig. 1 without making it too complex is unclear. Here, objects are instantiated from the first to the second column. We have not provided details about the parent objects, as we believe these details are not important for reader comprehension. In its present form, the objects are inherited from Pandas objects, but it is possible that a future version is based on something else. For the users, this will be transparent as the toolbox is designed in such a way that only the methods that are specific to Pynapple are needed to do most computation, while only expert programmers may be interested in using Pandas functionalities.

- *The authors wrote that Pynapple does -*

"not depend on any external package"

As mentioned above, this is not true. It depends on Numpy and likely other packages, and this should be explained. It is perfectly reasonable to say that it depends on only a few other packages.

As said above, we have now clarified this claim.

Page 5.

- *The authors wrote -*

"represent arrays of Ts and Tsd"

For a knowledgeable reader's reference, it would be helpful to refer to these either as Numpy arrays (at least at first when they are defined) or as lists if they are native python objects.

Indeed, using the word "arrays" here could be confusing because of Numpy arrays. We have changed this term with "groups".

- *The authors wrote -*

"Pynapple is built with objects from the Pandas library ... Pynapple objects inherit the computational stability and flexibility"

Here a definition of stability would be useful. Is it the case that by stability you mean "does not change over time"? Or is some other meaning of stability implied?

Yes, this is exactly what we meant when referring to the stability of Pandas. We have added the following precision:

"As such, Pynapple objects inherit the long-term consistency of the code and the computational flexibility computational stability and flexibility from this widely used package."

Page 6

- *Fig 2*

In Fig 2 A and B, the illustrations are good. It would also be very helpful to use toy code examples to illustrate how Pynapple will be used to carry out on a sample analysis-problem so that potential users can see what would need to be done.

We appreciate the kind works. Regarding the toy code, this is what we tried to do in Fig. 4. Instead of including the code directly in the paper, which does not seem a modern way of doing this, we now refer to the online notebooks that reproduce all panels of Figure 4.

- *The authors wrote -*

"While these objects and methods are relatively few"

In object-oriented programming, objects contain methods. If a method is not in an object, it is not technically a method but a function. It would be helpful if the authors made sure their terminology is accurate, perhaps by saying something like "While there are relatively few objects, and while each object has relatively few methods ... "

We agree with the referee, we have changed the sentence accordingly.

- *The authors wrote -*

"if not implemented correctly, they can be both computationally intensive and highly susceptible to user error"

Here the authors are using "correctly" to refer to two things - "accuracy" - gesng the right answer, and "efficiency" - gesng to that answer with relatively less computation. It would be clearer if they split out those two concepts in the phrasing.

Indeed, we used the term to cover both aspects of the problem, leading to the two possible issues cited in the second part of the sentence. We have changed the sentence following the referee's advice:

"While there are relatively few objects, and while each object has relatively few methods, they are the foundation of almost any analysis in systems neuroscience. However, if not implemented efficiently, they can be computationally intensive and if not implemented accurately, they are highly susceptible to user error."

- *In the next sentence the authors wrote -*

"Pynapple addresses this concern."

This statement would benefit from just additional text explaining how the concern is addressed.

We thank the referee for the suggestion. We have changed the sentence to this one: "The implementation of core features in Pynapple addresses the concerns of efficiency and accuracy"

Page 9

- *The authors wrote -*

This is implemented via a set of specialized object subclasses of the BaseLoader class. To avoid code redundancy, these I/O classes inherit the properties of the BaseLoader class. "

From a programming perspective, the point of a base class is to avoid redundancy, so it might be better to just mention that this avoids the need to redefine I/O operations in each class.

We have rephrased the sentence as follows:

"This is implemented via a set of specialized object subclasses of the BaseLoader class, avoiding the need to redefine I/O operations in each subclass"

- *The authors wrote -*

"classes are unique and independent from each other, ensuring stability"

How do classes being unique and independent ensure stability? Perhaps here again the misunderstanding is due to the lack of a definition of stability.

We thank the referee for the remark. We first changed “stability” for “long-term backward compatibility”. We further added the following sentence to clarify this claim. “For instance, if the spike sorting tool Phy changes its output in the future, this would not affect the “Neurosuite” IO class as they are independent of each other. This allows each tool to be updated or modified independently, without requiring changes to the other tool or the overall data format.”

- *The authors wrote -*

"Using preexisting code to load data in a specific manner instead of rewriting already existing functions avoids preprocessing errors"

Here it might be helpful to use the lingo of Object-oriented programming. (e.g. inheritance and polymorphism). Defining these terms for a neuroscience audience would be useful as well.

We do not think it is necessary to use too much technical term in this manuscript. However, this sentence was indeed confusing. We have now simplified it:

"[...], users can develop their own custom I/O using available template classes. Pynapple already includes several of such templates and we expect this collection to grow in the future."

Page 10

- *The authors wrote -*

"These analyses are powerful because they are able to describe the relationships between time series objects while requiring the fewest number of parameters to be set by the user."

It is not clear that this makes for a powerful analysis as opposed to an easy-to-use analysis.

We have changed “powerful” with “easy to use”.

Page 12

"they are built-in and thus do not have any external dependencies"

If the authors want to retain this, it would be helpful to explain (perhaps in the introduction) why having fewer external dependencies is useful. And is it true that these functions use only base python classes?

We have rephrased this sentence as follows:

“they are for the most part built-in and only depend on a few common external packages, ensuring that they can be used stand-alone without relying on packages that are at risk of not being maintained or of not being compatible in the near future.”

Other comments:

- It would be helpful, as mentioned in the public review, to frame this work in the broader context of what is needed to go from data to scientific results so that people understand what this package does and does not provide.*

We have added the following sentence to the discussion to make sure readers understand:

“The path from data collection to reliable results involves a number of critical steps: exploratory data analysis, development of an analysis pipeline that can involve custom-made developed processing steps, and ideally the use of that pipeline and others to replicate the results. Pynapple provides a platform for these steps.”

- It would also be helpful to describe the Pynapple so<ware ecosystem as something that readers could contribute to. Note here that GNU may not be a good license. Technically, GNU requires any changes users make to Pynapple for their internal needs to be offered back to the Pynapple team. Some labs may find that burdensome or unacceptable. A workaround would be to have GNU and MIT licenses.*

The main restriction of the GPL license is that if the code is changed by others and released, a similar license should be used, so that it cannot become proprietary. We therefore stick to this choice of license.

We would be more than happy to receive contributions from the community. To note, several users outside the lab have already contributed. We have added the following sentence in the introduction:

“As all users are also invited to contribute to the Pynapple ecosystem, this framework also provides a foundation upon which novel analyses can be shared and collectively built by the neuroscience community.”

- This so<ware shares some similarities with the nelpy package, and some mention of that package would be appropriate.*

While we acknowledge the reviewer’s observation that Nelpy is a similar package to Pynapple, there are several important differences between the two.

First, Nelpy includes predefined objects such as SpikeTrain, BinnedSpikeTrain, and AnalogSignal, whereas Pynapple would use only Ts and Tsd for those. This design choice was

made to provide greater flexibility and allow users to define their own data structures as needed.

Second, Nelpy is primarily focused on electrophysiology data, whereas Pynapple is designed to handle a wider range of data types, including calcium imaging and behavioral data. This reflects our belief that the NWB format should be able to accommodate diverse experimental paradigms and modalities.

Finally, while Nelpy offers visualization and high-level analysis tools tailored to electrophysiology, Pynapple takes a more general-purpose approach. We believe that users should be free to choose their own visualization and analysis tools based on their specific needs and preferences.

The package has now been cited.

Reviewer #2 (Public Review):

Pynapple and Pynacollada have the potential to become very valuable and foundational tools for the analysis of neurophysiological data. NWB still has a steep learning curve and Pynapple offers a user-friendly toolset that can also serve as a wrapper for NWB.

The scope of the manuscript is not clear to me, and the authors could help clarify if Pynacollada and other toolsets in the making become a future aspect of this paper (and Pynapple), or are the authors planning on building these as separate publications.

The author writes that Pynapple can be used without the I/O layer, but the author should clarify how or if Pynapple may work outside NWB.

Absolutely. Pynapple can be used for generic data analysis, with no requirement of specific inputs nor NWB data. For example, the lab is currently using it for a computational project in which the data are loaded from simple files (and not from full I/O functions as provided in the toolbox) for further analysis and figure generation.

This was already noted in the manuscript, last paragraph of the section “Importing data from common and custom pipelines”

“Third, users can still use Pynapple without using the I/O layer of Pynapple.”

We have added the following sentence in the discussion

“To note, Pynapple can be used without the I/O layer and independent of NWB for generic, on-the-fly analysis of data.”

This brings us to an important fundamental question. What are the advantages of the current approach, where data is imported into the Ts objects, compared to doing the data import into NWB files directly, and then making Pynapple secondary objects loaded from the NWB file? Does NWB natively have the ability to store the 5 object types or are they initialized on every load call?

NWB and Pynapple are complimentary but not interdependent. NWB is meant to ensure long-term storage of data and as such contains as much information as possible to describe the experiment. Pynapple does not use NWB to directly store the objects, however it can read from NWB to organize the data in Pynapple objects. Since the original version of this manuscript was submitted, new methods address this. Specifically, in the current beta version, each object now has a “save” method. Obviously, we are developing functions to load these objects as well. This does not depend on NWB but on npz, a Numpy specific file format.

However, we believe it is a bit too premature to include these recent developments in the manuscript and prefer not to discuss this for now.

Many of these functions and objects have a long history in MATLAB - which documents their usefulness, and I believe it would be fitting to put further stress on this aspect - what aspects already existed in MATLAB and what is completely novel. A widely used MATLAB toolset, the FMA toolbox (the Freely moving animal toolbox) has not been cited, which I believe is a mistake.

We agree that the FMA toolbox should have been cited. This has now been corrected.

Pynapple was first developed in Matlab (it was then called TSToolbox). The first advantage is of course that Python is more accessible than Matlab. It has also been adopted by a large community of developers in data analysis and signal processing, which has become without a doubt much larger than the Matlab community, making it possible to find solutions online for virtually any problem one can have. Furthermore, in our experience, trainees are now unwilling to get training in Matlab.

Yet, Python has drawbacks, which we are fully aware of. Matlab can be very computationally efficient, and old code can usually run without any change, even many years later.

A limitation in using NWB files is its standardization with limited built-in options for derived data and additional metadata. How are derived data stored in the NWB files?

NWB has predetermined a certain number of data containers, which are most common in systems neuroscience. It is theoretically possible to store any kind of data and associated metadata in NWB but this is difficult for a non-expert user. In addition, NWB does not allow data replacement, making it necessary to rewrite a whole new NWB file each time derived data are changed and stored. Therefore, we are currently addressing this issue as described above. Derived data and metadata will soon be easy to store and read.

How is Pynapple handling an existing NWB dataset, where spikes, behavioral traces, and other data types have already been imported?

This is an interesting point. In theory, Pynapple should be able to open a NWB file automatically, without providing much information. In fact, it is challenging to open a NWB file without knowing what to look for exactly and how the data were preprocessed. This would require adapting a I/O function for a specific NWB file. Unfortunately, we do not believe there is a universal solution to this problem. There are solutions being developed by others, for example NWB Widgets (NWB Widgets). We will keep an eye on this and see whether this could be adapted to create a universal NWB loader for Pynapple.

Reviewer #2 (Recommendations For The Authors):

Other tools and solutions are being developed by the NWB community. How will you make sure that these tools can take advantage of Pynapple and vice versa?

We recognize the importance of collaboration within the NWB community and are committed to making sure that our tools can integrate seamlessly with other tools and solutions developed by the community.

Regarding Pynapple specifically, we are designing it to be modular and flexible, with clear APIs and documentation, so that other tools can easily interface with it. One important thing is that we want to make sure Pynapple is not too dependent of another package or file format

such as NWB. Ideally, Pynapple should be designed so that it is independent of the underlying data storage pipeline.

Most of the tools that have been developed in the NWB community so far were designed for data visualisation and data conversion, something that Pynapple does not currently address. Multiple packages for behavioral analysis and exploration of electro/optophysiological datasets are compatible with the NWB format but do not provide additional solutions per se. They are complementary to Pynapple.